Hardware-in-the-Loop Verification of Distance-3 Surface-Code Quantum Memory on a 28-Qubit HBM-Accelerated FPGA Statevector Simulator

Nasir Ali[1*]

[1*]Embedded Systems Division, Centre for Development of Advanced Computing (C-DAC), Noida, 201307, Uttar Pradesh, India.

Corresponding author(s). E-mail(s): nasirali2607@gmail.com;

Abstract

Surface codes are the leading practical route to fault-tolerant quantum computation, but validating the classical control and decoding stack that a surface-code memory needs – repeated syndrome extraction, ancilla reset, and syndrome-history decoding – ordinarily requires either scarce physical qubits or a Clifford-only stabilizer simulator that discards the full quantum state. This paper reports an implementation of a distance-3 rotated surface code (9 data and 8 ancilla qubits, of 28 addressable) on an HBM-resident, complex128 FPGA statevector accelerator (AMD/Xilinx Alveo U55C), and shows how to run a genuine multi-round fault-tolerant memory experiment on hardware that – like many first-generation statevector accelerators – exposes no native mid-circuit measurement-and-branch primitive. Every entangling operation (synthetic error injection, ancilla reset, and the measure-$\boldsymbol{Z}$/measure-$\boldsymbol{X}$ syndrome-extraction circuits) executes as a real unitary circuit on the FPGA kernel each round; only the unavoidable measurement collapse and the classical minimum-weight-perfect-matching (MWPM) decoding run on the host, mirroring the classical/quantum division of labour the theory itself prescribes. The full stabilizer set, logical-operator pair, and decoding graph were derived from first principles and independently verified against `qiskit.quantum_info` (93 algebraic and decoding-logic assertions, all passing); every hardware trial additionally cross-checks the FPGA-computed statevector's logical readout against an independently tracked classical Pauli frame, catching a real decoding-graph connectivity bug during development. Across a physical-error-rate sweep on real hardware (150 trials, 600 FPGA-executed syndrome rounds, zero cross-validation discrepancies), the decoded logical error rate is consistent with zero at $\boldsymbol{p = 1\%}$ and rises sharply above it, in qualitative agreement with the accuracy threshold

reported by Dennis, Kitaev, Landahl and Preskill. We discuss the engineering trade-offs of the host-mediated protocol, a decoding-graph construction pitfall we found and fixed, and the throughput gap between this exact-statevector validation approach and dedicated real-time surface-code decoder pipelines.

Keywords: quantum error correction, surface codes, FPGA acceleration, statevector simulation, HBM, minimum-weight perfect matching, syndrome extraction, embedded quantum control systems

1 Introduction

Surface codes [1–4] are, by consensus, the most experimentally mature route to fault-tolerant quantum computation: their stabilizer generators act only on geometrically local groups of qubits, they tolerate physical error rates on the order of 10^{-2}–10^{-3}, and superconducting-qubit demonstrations have now shown logical error suppression that scales as the code distance grows [7]. All of these demonstrations, however, depend on a substantial classical control and decoding stack: syndrome bits must be extracted every cycle, ancillas reset, a running syndrome history assembled into a space-time detector graph, and a minimum-weight matching (or equivalent) decoder must return a correction fast enough to keep pace with the physical cycle time. Developing and validating that classical stack does not, in principle, require physical qubits at all – but it does require a simulation substrate that (a) reproduces the true quantum state closely enough to serve as ground truth, and (b) is realised on hardware whose execution model forces the same architectural choices a real control system must make.

Two families of tools sit at the extremes of this design space. Stabilizer-only simulators such as Stim [12] are extremely fast because they never materialise a 2^n-dimensional state vector, but that speed comes from restricting the simulated circuit to the Clifford group – adequate for most surface-code memory studies, but not for future work that couples QEC circuits to non-Clifford resource states, coherent errors, or leakage. At the other extreme, full statevector simulation retains the exact quantum state at the cost of exponential memory, which is why FPGA- and GPU-accelerated statevector engines with high-bandwidth memory (HBM) have become an active area of embedded/accelerator systems research, including our own prior work on FPGA-resident statevector, density-matrix and matrix-product-state engines for the same Alveo U55C hardware family [17–19]. What has not, to our knowledge, been previously reported is a full multi-round surface-code syndrome-extraction-and-decode loop executed on such a statevector accelerator, with an explicit protocol for the case – common to many first-generation FPGA/GPU statevector kernels, including the one used here and in a companion real-time-decoder study [20] – where the accelerator itself offers no native mid-circuit measurement-and-classically-conditioned-branch instruction.

This absence is not a minor implementation detail. A textbook description of repeated stabilizer measurement (e.g. Fowler et al.'s measure-Z/measure-X circuits [4]) implicitly assumes the device can measure an ancilla, branch on the outcome,

reset it, and continue – all within one coherent circuit. Many statevector kernels instead expose only (i) fully unitary gate sequences and (ii) an end-of-circuit, Born-rule sample of the final state; a hardware `reset` instruction, where present, typically performs a deterministic projection rather than returning a classical outcome a subsequent gate could condition on. Section 5 shows how to build a faithful, verifiable multi-round QEC memory experiment on top of exactly these two primitives, by dispatching one purely unitary circuit per syndrome round to the accelerator and performing only the physically unavoidable measurement collapse and the classical decoding on the host – which is, in fact, precisely the division of labour the surface-code fault-tolerance literature itself prescribes for the quantum device versus its classical controller [3].

1.1 Contributions

1. A distance-3 rotated surface code (9 data qubits, 8 ancilla qubits, 17 of 28 addressable qubits) whose stabilizer generators, logical operator pair and decoding-graph boundary connectivity are derived from first principles – not copied from a published figure – and independently verified against an exact linear-algebra reference (`qiskit.quantum_info`), reported as a reusable derivation method in Section 4.
2. A host/FPGA co-execution protocol for multi-round syndrome extraction on an accelerator with no native mid-circuit classical feedback (Section 5), in which every entangling and error-injection operation still executes on the FPGA kernel and only measurement collapse and decoding run on the host.
3. A synthetic-syndrome-data generator implementing the Dennis–Kitaev–Landahl–Preskill error model [3], in which qubit bit-flip errors, qubit phase-flip errors and syndrome-readout errors all occur at the same rate p, giving controlled, repeatable QEC benchmarking without physical qubits.
4. A from-scratch space-time minimum-weight-matching decoder, exact for the small instances this code produces, together with a documented decoding-graph construction pitfall we found and fixed during development (Section 6) – restricting boundary connectivity to weight-2 "boundary" stabilizers only, which underconnects the graph and lets certain weight-1 physical errors decode to a logical error, violating the distance-3 guarantee.
5. An end-to-end hardware/software cross-validation methodology in which every trial's FPGA-computed statevector logical readout is checked against an independently tracked classical Pauli frame, giving a built-in correctness oracle for the whole pipeline (Section 8).
6. Experimental results on real Alveo U55C hardware across a physical-error-rate sweep (Section 8), reproducing the qualitative threshold behaviour reported in [3].

The remainder of the paper is organised as follows. Section 2 surveys related work in surface-code theory, decoding, and FPGA/GPU-accelerated quantum simulation. Section 3 summarises the surface-code formalism and the FPGA statevector accelerator's architecture and execution model. Section 4 derives and verifies the distance-3 code instance used throughout. Section 5 presents the host/FPGA co-execution protocol and synthetic-syndrome-data generator. Section 6 describes the

space-time decoding graph and the matching decoder. Section 7 details the experimental setup, and Section 8 reports and discusses the results. Section 10 concludes and outlines future work.

2 Related work

Surface-code theory. Kitaev's toric code [1] and its planar (boundary-truncated) variants [2] underlie the modern surface code. Dennis, Kitaev, Landahl and Preskill [3] formalised the accuracy-threshold analysis this paper's error model follows, including the observation that qubit bit-flip errors, qubit phase-flip errors and syndrome-measurement errors can be treated at a common probability p for threshold estimation purposes. Fowler, Mariantoni, Martinis and Cleland [4] gave the concrete measure-Z/measure-X ancilla circuits this paper implements verbatim, and estimated physical-qubit overheads for large-scale factoring circuits. Tomita and Svore [5] studied small-distance rotated surface codes under realistic circuit-level noise and identified the hook-error-optimal CNOT scheduling that Section 4 explicitly does *not* implement (a scoped simplification discussed in Section 9). Versluis et al. [6] describe a scalable control architecture for a distance-3 rotated surface code on superconducting hardware; the lattice used here was derived independently from first principles (Section 4) rather than reproduced from that or any other single reference. Google Quantum AI's 2023 demonstration [7] showed logical error suppression with increasing code distance on real superconducting hardware, the experimental benchmark against which classical-control simulation work such as this paper is ultimately positioned. IBM's heavy-hexagon subsystem-code work [8] is a further example of hardware-aware code design trading stabilizer weight for connectivity constraints, a consideration directly relevant to any future port of this methodology to a different accelerator topology.

Decoding. Minimum-weight perfect matching traces to Edmonds' blossom algorithm [9]; Higgott's PyMatching [10] is the de facto standard practical implementation for surface-code decoding graphs at scale, and Delfosse and Nickerson's Union-Find decoder [11] targets near-linear-time decoding for real-time use. This paper implements its own small, exact (brute-force, boundary-augmented) MWPM solver rather than depending on an external decoding library, since the distance-3 instance studied here produces at most a handful of simultaneous detection events per trial (Section 6); adopting PyMatching or a Union-Find decoder is identified as necessary future work for larger code distances (Section 9).

Classical simulation of QEC circuits. Stim [12] simulates stabilizer circuits at very high throughput but is restricted to the Clifford group, which this paper's synthetic-error-injection circuits happen to respect but which a full statevector engine is not restricted to; the value of the present approach is precisely that it does not rely on that restriction, at the cost of the exponential memory a statevector representation requires.

FPGA/GPU-accelerated quantum simulation. This work reuses the HBM-resident, complex128 statevector kernel architecture described in a companion submission [17], part of a broader indigenous FPGA quantum-simulation effort at C-DAC that also includes a 30-qubit statevector engine [18], an FPGA matrix-product-state

engine [19], and a dedicated real-time LUT/Union-Find/MWPM decoder pipeline targeting sub-microsecond syndrome-to-correction latency on the same Alveo U55C hardware family [20]. That decoder pipeline and the present paper are complementary rather than overlapping: the former targets decoding *latency* on physically streamed syndrome data for Shor and Steane codes, whereas this paper targets *algorithmic correctness validation* of a full surface-code memory protocol against an exact simulated quantum state, at wall-clock timescales orders of magnitude slower than a real decoding cycle (Section 9). NVIDIA's CUDA-Q / CUDA-QX (`cudaq-qec`) toolkit represents a GPU-accelerated approach to the same broader problem space and is noted here as related, complementary tooling rather than a baseline this paper benchmarks against.

To the best of our knowledge, no prior work reports a full multi-round surface-code syndrome-extraction-and-decode loop, cross-validated against an exact statevector, on FPGA-accelerated hardware whose execution model lacks native mid-circuit classical feedback, together with an explicit, reusable protocol for that constraint. That is the gap this paper addresses.

3 Background

3.1 Rotated surface codes and the accuracy threshold

A distance-d rotated surface code encodes one logical qubit into $2d^2-1$ physical qubits: d^2 data qubits carrying the code's Pauli-X/Z degrees of freedom, and d^2-1 ancilla qubits, each measuring one stabilizer generator. Stabilizers come in two flavours – X-type, $\hat{X}_a\hat{X}_b\hat{X}_c\hat{X}_d$ (weight 4 in the bulk, weight 2 at one pair of boundaries), and Z-type, $\hat{Z}_a\hat{Z}_b\hat{Z}_c\hat{Z}_d$ (weight 4 in the bulk, weight 2 at the other pair of boundaries) – chosen so that every X-type and every Z-type generator commutes (even qubit-support overlap), while the code's two logical operators $\bar{X}$, $\bar{Z}$ each commute with every stabilizer and anticommute with each other. Physical Pauli errors that anticommute with a stabilizer flip its measured eigenvalue (a *detection event*); a classical decoder infers a correction from the pattern of detection events, typically by minimum-weight matching on a graph whose edges represent the physical errors most likely to explain a given syndrome [3, 4].

Dennis, Kitaev, Landahl and Preskill [3] showed that, under a model in which qubit bit-flip errors, qubit phase-flip errors and syndrome-measurement errors all occur independently with the same probability p per qubit per round, there is an accuracy threshold p_c (estimated at $p_c \gtrsim 1.14\%$ for their optimal-recovery bound) below which increasing the code distance suppresses the logical error rate arbitrarily; above threshold, larger codes perform worse. This paper's error model and its "quantum memory" experiment design (encode, apply repeated noisy syndrome rounds, decode once, and check whether the logical value survived) follow that construction directly (Section 5).

3.2 The FPGA statevector accelerator

The target platform is an AMD/Xilinx Alveo U55C accelerator card hosting a custom HLS kernel (`quantum_fpga_kernel_complex128`, compiled from a C++ HLS source

via Vitis) that maintains a 2^n-amplitude complex128 statevector across two HBM-resident arrays – one for the real part, one for the imaginary part – each striped across 8 HBM pseudo-channels (256 MB each, 2 GB per array). With 2^n amplitudes $\times$ 8 bytes (double) = 2 GB exactly at $n = 28$, the kernel's HBM allocation caps addressable circuits at MAX_QUBITS_FPGA = 28 without widening the per-array HBM range. A third, smaller HBM-resident buffer carries the gate stream: each gate is encoded as an 8-word record (opcode, up to three qubit indices, and up to three packed single-precision parameters), and up to 512 gates are dispatched per kernel launch.

The kernel natively executes 19 opcodes (identity, the seven single-qubit Clifford/Pauli gates $H, X, Y, Z, S, S^\dagger, \sqrt{X}, \sqrt{X}^\dagger$, $T/T^\dagger$, the parametrised rotations $R_x, R_y, R_z, P, U_2, U_3$, and the two-qubit entanglers CX and CZ). A host-side lowering pass decomposes every other supported gate (SWAP, controlled-$Y/H/R_x/R_y/R_z/\sqrt{X}$, DCX, iSWAP, ECR, $R_{xx}/R_{yy}/R_{zz}$, Toffoli, and general multi-controlled-X) into this native basis using standard circuit identities before dispatch, so that every circuit the host driver accepts ultimately reduces to single-qubit rotations plus CX/CZ on the kernel itself.

Two properties of this execution model shape the protocol design in Section 5. First, a `reset` instruction deterministically projects a qubit onto $|0\rangle$ (zeroing the amplitudes with that qubit set to 1 and renormalising) rather than returning a classical outcome; second, `measure` instructions are resolved only at the end of a circuit, by Born-rule sampling of the final statevector, and the driver returns the exact pre-sampling statevector alongside the sampled bitstrings. There is no instruction that measures a qubit *mid-circuit* and conditions a subsequent gate on the outcome. This is a common characteristic of first-generation FPGA/GPU statevector kernels (including our own prior work [17, 18]), and is the constraint Section 5 is designed around, rather than a limitation specific to surface-code circuits.

The host driver exposes this kernel through a Qiskit-compatible Python API over AMD's Xilinx Runtime (XRT); circuits are supplied as gate tuples, executed via one or more kernel launches per circuit, and returned as the exact final statevector together with Born-rule-sampled measurement outcomes.

4 Distance-3 rotated surface code: derivation and verification

4.1 Lattice derivation

Rather than reproduce a published lattice diagram, the code instance used throughout was derived directly from the stabilizer-commutation requirement, as a reusable method applicable to any code distance within a target qubit budget. Nine data qubits $q_0, \ldots, q_8$ are placed on a 3×3 grid, indexed q_{3y+x} for $(x, y) \in \{0, 1, 2\}^2$. Candidate ancilla positions are placed at every half-integer coordinate $(x + \frac{1}{2}, y + \frac{1}{2})$ for $x, y \in \{-1, 0, 1, 2\}$ (one ring of padding around the data grid), and a candidate is discarded if it neighbours fewer than two data qubits (the four lattice corners). Each surviving candidate is coloured by the parity of $x + y$: one colour becomes an X-type

Table 1 Stabilizer generators and logical operators of the distance-3 rotated surface code used in this work. Ancilla qubit indices: $\mathrm{XS}_{0\text{–}3} \mapsto q_{9\text{–}12}$, $\mathrm{ZS}_{0\text{–}3} \mapsto q_{13\text{–}16}$.

Generator	Support	Type	Boundary
XS_0	q_0, q_1	weight 2	top
XS_1	q_1, q_2, q_4, q_5	weight 4	interior
XS_2	q_3, q_4, q_6, q_7	weight 4	interior
XS_3	q_7, q_8	weight 2	bottom
ZS_0	q_0, q_1, q_3, q_4	weight 4	interior
ZS_1	q_4, q_5, q_7, q_8	weight 4	interior
ZS_2	q_3, q_6	weight 2	left
ZS_3	q_2, q_5	weight 2	right
$\bar{X}$ (logical X)	q_0, q_3, q_6	left column	
$\bar{Z}$ (logical Z)	q_0, q_1, q_2	top row	

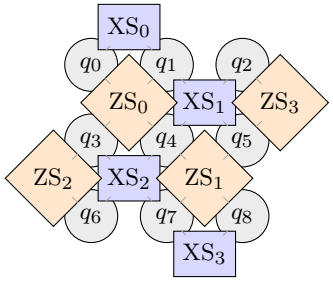


Fig. 1 Distance-3 rotated surface code layout. Grey circles: data qubits (q_0–q_8). Blue squares: X-type ancillas (detect Z errors, truncated to weight 2 at the top/bottom boundary). Orange diamonds: Z-type ancillas (detect X errors, truncated to weight 2 at the left/right boundary). $\bar{X}$ runs down the left column (q_0, q_3, q_6); $\bar{Z}$ runs across the top row (q_0, q_1, q_2).

ancilla, the other Z-type. At each of the four physical edges, exactly one of the two colour-alternating boundary candidates is kept, chosen so that X-type ancillas are truncated to weight 2 on the top/bottom edges and Z-type ancillas on the left/right edges. This yields the eight stabilizer generators in Table 1 and the qubit layout in Figure 1: four X-type (detecting Z errors) and four Z-type (detecting X errors), for $9 + 8 = 17$ of the 28 available qubits.

4.2 Independent verification

Because the lattice was derived rather than transcribed, it was verified independently of the FPGA circuit implementation using `qiskit.quantum_info`, before any hardware was touched. The verification checks, run as a pre-flight self-test ahead of every hardware invocation:

- every pair of same-type stabilizers commutes (trivially true for same-Pauli-letter operators, checked regardless);
- every X-type stabilizer commutes with every Z-type stabilizer (even support overlap);

- $\bar{X}$ and $\bar{Z}$ each commute with all eight stabilizers, and anticommute with each other;
- the code state obtained by projecting $|0\rangle^{\otimes 9}$ onto the joint $+1$ eigenspace of the four X-stabilizers (via explicit operator projectors $\frac{1}{2}(I + \mathrm{XS}_i)$ applied in sequence) has expectation value $+1$ for every one of the eight stabilizers and for $\bar{Z}$;
- every possible single-qubit Z error, and every possible single-qubit X error, decodes (Section 6) to a correction that leaves the corresponding logical operator's value unchanged.

This amounts to 93 independent assertions, all of which pass (Section 8). The final class of check – exhaustive single-qubit-error decoding – caught a real defect in an early version of the decoding-graph construction, discussed as a case study in Section 6.

5 Host/FPGA co-execution protocol

5.1 Design

Given the execution model of Section 3.2, a syndrome round cannot be expressed as "measure the ancillas, branch, reset" within a single kernel invocation. Instead, each round is dispatched to the FPGA as one purely unitary circuit, and the measurement that a real device would perform in hardware is instead performed on the host, by exact Born-rule sampling of the statevector the FPGA kernel returns – mathematically identical to what a physical device's own classical control system would compute from its measurement record, just executed between kernel launches rather than inside one. Algorithm 1 summarises one round; Figure 2 shows the resulting host/FPGA data flow.

The CNOTs within a single stabilizer are scheduled in a simple, deterministic top-to-bottom, left-to-right order. Any ordering of commuting-target CNOTs onto the same ancilla yields an identical final parity, so this ordering is functionally correct; it is not, however, the hook-error-optimal NW/NE/SW/SE schedule of [4, 5], which matters for circuit-level noise injected *during* the entangling gates. Because this study's noise model injects errors between rounds rather than mid-gate (Section 5.2), hook-error propagation is not exercised here; adopting the optimal schedule is noted as future work (Section 9).

5.2 Synthetic syndrome data generation

Following [3], three independent error channels share one probability p per round: (i) a bit-flip (X) error on each data qubit with probability p; (ii) a phase-flip (Z) error on each data qubit with probability p; and (iii) a bit-flip on each *recorded* syndrome bit with probability q_{meas} (default $q_{\mathrm{meas}} = p$), applied only to the classical record the decoder sees, never to the physical ancilla state used to drive the next round's reset – this distinguishes a genuine measurement error (a misread of a correctly prepared ancilla) from a physical qubit error. The final round of each trial is recorded without measurement-bit-flip noise, the standard time-boundary closure that lets a finite-round decoding graph terminate cleanly [3, 4]. This construction gives a fully synthetic, reproducible syndrome history – no physical qubits are involved anywhere

Algorithm 1 One syndrome-extraction round (host-orchestrated, FPGA-executed)

Require: current 17-qubit statevector $|\psi\rangle$; ancillas that read 1 last round, R_X, R_Z

$G \leftarrow \{\}$
for $a \in R_X \cup R_Z$ **do** ▷ physically reset last round's ancillas
  $G \leftarrow G \cup \{X(a)\}$
end for
for $d \in$ data qubits **do**
  with prob. p: $G \leftarrow G \cup \{X(d)\}$; with prob. p: $G \leftarrow G \cup \{Z(d)\}$ ▷ synthetic error
end for
for each Z-stabilizer with support S, ancilla a **do**
  $G \leftarrow G \cup \{\mathrm{CX}(d, a) : d \in S\}$ ▷ measure-Z, Fig. 1b of [4]
end for
for each X-stabilizer with support S, ancilla a **do**
  $G \leftarrow G \cup \{H(a)\} \cup \{\mathrm{CX}(a, d) : d \in S\} \cup \{H(a)\}$ ▷ measure-X, Fig. 1c of [4]
end for
$|\psi'\rangle \leftarrow \textsc{FpgaRunCircuit}(|\psi\rangle, G)$ ▷ exact statevector, real hardware
$b \leftarrow \textsc{SampleBornOutcome}(|\psi'\rangle, \text{ancilla qubits})$ ▷ host, exact Born-rule sampling
$|\psi''\rangle \leftarrow \textsc{ProjectAndRenormalise}(|\psi'\rangle, b)$
$R_X \leftarrow \{a \in X\text{-ancillas} : b_a = 1\}$; $R_Z \leftarrow \{a \in Z\text{-ancillas} : b_a = 1\}$
return $|\psi''\rangle$, syndrome bits b (optionally corrupted per Section 5.2), R_X, R_Z

in the pipeline – while remaining a faithful re-implementation of the error model the accuracy-threshold literature analyses.

The experiment tracks a single logical observable per run (by default $\bar{Z}$, i.e. a $|0\rangle_L$ memory protected against the accumulated X-error channel via the Z-stabilizer graph); an $\bar{X}/|+\rangle_L$ run is obtained symmetrically by Hadamard-preparing the data register and decoding via the X-stabilizer graph instead.

6 Decoding: space-time matching graph

6.1 Graph construction

For each stabilizer type, a space-time graph is built with one node per (stabilizer, round) pair. Spatial edges (weight 1) connect two same-round, same-type stabilizers whose supports share exactly one data qubit – the qubit that a single physical error on it would flip both eigenvalues of. Temporal edges (weight 1) connect the same stabilizer in consecutive rounds, representing a syndrome *measurement* error rather than a data-qubit error; a matched pair of detection events joined only by temporal edges therefore contributes no physical-qubit correction. Boundary edges connect a stabilizer to one shared virtual boundary node.

The correct, general rule for boundary edges – and the one this paper's implementation ultimately uses – is topological rather than based on stabilizer weight: each data qubit belongs to at most two same-type stabilizers by construction (an invariant checked programmatically); a qubit belonging to exactly two is already represented

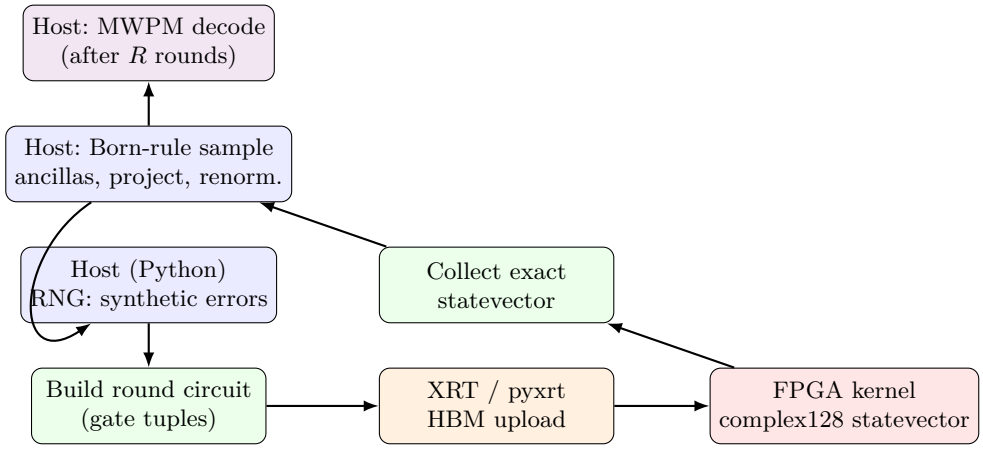


Fig. 2 Host/FPGA co-execution loop for one syndrome round. Every entangling operation (error injection, ancilla reset, measure-Z/measure-X circuits) runs on the FPGA kernel; only Born-rule sampling, projection, and the classical MWPM decoder run on the host.

by the spatial edge between them, while a qubit belonging to exactly one has no same-round partner and must connect *directly* to the boundary from its sole owner, regardless of that stabilizer's weight.

6.2 A decoding-graph pitfall, found by the self-test

An earlier version of this code restricted boundary edges to weight-2 ("boundary-type") stabilizers only, on the reasoning that those are the ones the lattice derivation of Section 4 explicitly truncates at a physical edge. This is wrong: two of the four interior, weight-4 stabilizers (XS_1 and XS_2 in Table 1) each own a data qubit (q_2, q_5 and q_3, q_6 respectively) that no neighbouring stabilizer shares, because the lattice derivation keeps only one of the two colour-alternating boundary candidates at each physical edge (Section 4). Under the restricted rule, a single physical Z error on, e.g., q_2 produces a lone detection event at XS_1 with no direct boundary edge; the decoder was then forced to route through a neighbour (XS_0 →boundary), returning a correction on $\{q_0, q_1\}$ that leaves a residual operator with *odd* overlap with $\bar{X}$'s support – a logical error induced by a single weight-1 physical error, which directly violates the distance-3 guarantee that every weight-1 error is uniquely (or at least logically-equivalently) correctable.

This was caught, not by inspection, but by the exhaustive single-qubit-error sweep in the self-test (Section 4.2): four of nine single-Z-error cases failed their logical-preservation check under the restricted rule. Generalising the boundary rule to the topological (ownership-degree) criterion above – rather than the weight-based one – resolved all 93 self-test assertions (Section 8). We report this as a general lesson for anyone implementing a matching-graph decoder for a new stabilizer code by hand: boundary connectivity should be derived from qubit-ownership degree in the stabilizer-support graph, not from stabilizer weight, and an exhaustive single-error

sweep against an independent (non-graph-based) success criterion is an effective and cheap pre-deployment check.

We note that some qubits remain *syndrome-degenerate* even after this fix – a single Z error on q_2 and a single Z error on q_5 produce identical syndromes (both owned solely by XS_1) and are therefore indistinguishable to any decoder acting on the syndrome alone. This is expected code degeneracy, not a bug: because neither qubit lies in $\bar{X}$'s support, either choice of correction leaves the logical value unchanged, and the correct success criterion for a decoded correction is that the true error exclusive-ORed with the correction has even overlap with the relevant logical operator's support – not that it reproduces the physical error's exact location [3].

6.3 Matching

Detection events are collected across all R rounds by XOR-ing each round's recorded syndrome against the previous round's. The matching problem is solved exactly by brute-force enumeration over all perfect matchings of the detection-event set, augmented with one virtual boundary-node copy per event (so every event can independently match "to the boundary"), using precomputed breadth-first-search distances over the unweighted space-time graph. This is exact and fast at the event counts a distance-3 code produces at realistic p (typically 0–6 events per trial); a greedy nearest-neighbour fallback is used above 12 simultaneous events, which never triggered in this study's parameter range. Scaling to larger code distances would require replacing this solver with a library implementation such as PyMatching [10] or a Union-Find decoder [11] (Section 9).

7 Experimental setup

Hardware. A single AMD/Xilinx Alveo U55C accelerator card (shell `xilinx_u55c_gen3x16_xdma_base_3`), hosted in a Gigabyte G292-Z43 server (64 logical CPU cores, 256 GB RAM, Ubuntu 20.04.6 LTS, kernel 5.4.0), running Xilinx Runtime (XRT) 2.15.225. The kernel was built with Vitis HLS/`v++` with a 225 MHz clock target configured in the link stage; the post-route implementation report for this build recorded differing per-domain clocks (∼500 MHz kernel, ∼300 MHz data, ∼450 MHz HBM AXI), so all timings reported below are measured host-observed wall-clock times, not clock-cycle estimates.

Software. Python 3.8 host driver over `pyxrt`; NumPy 1.24 for host-side Born-rule sampling and state projection; Qiskit 1.2.4 used only for the independent self-test verification layer of Section 4.2, not for circuit execution.

Code and protocol. The distance-3 rotated surface code of Section 4 (17 of 28 available qubits), logical-$\bar{Z}$ memory experiment ($|0\rangle_L$ preparation, decoded against the accumulated X-error channel via the Z-stabilizer graph). $R = 4$ syndrome-extraction rounds per trial; $q_{\mathrm{meas}} = p$ except on the final round (Section 5.2); 30 independent trials per physical error rate; $p \in \{0.01, 0.03, 0.05, 0.08, 0.12\}$; pseudo-random seed fixed at 1234 for reproducibility.

Table 2 Logical error rate versus physical error rate p, $R = 4$ rounds, 30 trials each, distance-3 rotated surface code on real Alveo U55C hardware. The $p = 0.01$ row's upper bound is the exact (Clopper–Pearson) 95% confidence bound for zero events in 30 trials.

p	trials	logical failures	logical error rate	mean wall time/round
0.01	30	0	0.000 (≤ 0.095 at 95% CI)	353.6 ms
0.03	30	5	0.167	358.8 ms
0.05	30	8	0.267	369.7 ms
0.08	30	15	0.500	375.4 ms
0.12	30	14	0.467	389.1 ms

Metric. Logical error rate: the fraction of trials in which the decoded correction, exclusive-ORed with the true (classically tracked) accumulated error, has odd overlap with $\bar{Z}$'s support – i.e. the decoder failed to preserve the encoded logical value.

Built-in cross-validation. Because $\bar{Z}$ commutes with every stabilizer of both types by construction (Section 4), its value is provably unperturbed by stabilizer *measurement* and can only change under the accumulated physical X-error channel – meaning it is deterministic given that channel, independent of any stabilizer measurement randomness. Every trial therefore also samples the true final data-qubit register directly from the exact FPGA-returned statevector and compares the resulting logical bit against the independently tracked classical Pauli frame; a mismatch would raise an assertion and abort the run. This ran, silently, on every one of the 150 trials reported below.

8 Results

8.1 Static verification

All 93 self-test assertions of Section 4.2 pass: pairwise stabilizer commutation (48 checks), logical-operator commutation with every stabilizer and anticommutation with each other (17 checks), code-state stabilizer/logical eigenvalues (9 checks), and exhaustive single-qubit Z- and X-error decoding preserving the relevant logical operator (18 checks), plus one two-error independence check.

8.2 Hardware/software cross-validation

Across the full sweep (150 trials, 600 FPGA-executed syndrome rounds – 4 rounds $\times$ 30 trials $\times$ 5 physical error rates), zero discrepancies were observed between the FPGA-computed statevector's sampled logical readout and the independently tracked classical Pauli frame. This gives empirical, hardware-level confidence that the kernel's numerics, the host-side gate lowering, and the Born-rule sampling/projection implementation are all mutually consistent, in addition to the purely algebraic self-test of Section 4.2.

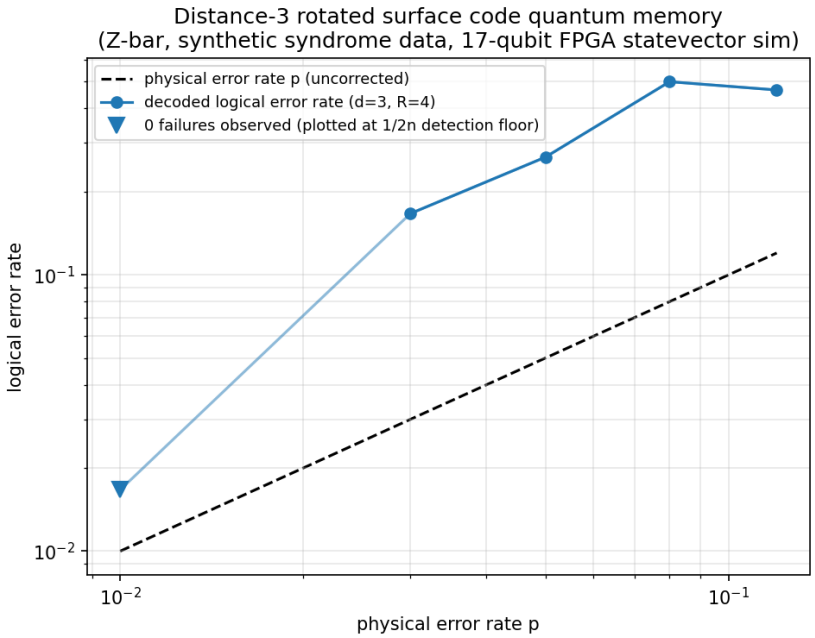


Fig. 3 Decoded logical error rate versus physical error rate p (distance-3, $R = 4$ rounds, 30 trials/point, real FPGA hardware). Dashed line: uncorrected physical error rate p for reference. The $p = 0.01$ point (triangle marker) is plotted at the $1/2n$ trial-limited detection floor, since zero failures were observed at that point.

8.3 Logical error rate versus physical error rate

Table 2 and Figure 3 summarise the sweep. At $p = 1\%$, zero logical failures were observed in 30 trials (exact Clopper–Pearson 95% upper bound $\approx 9.5\%$); by $p = 3\%$ the logical error rate has already risen to 16.7%, and continues rising (with one non-monotonic point at $p = 0.12$, within the sampling noise expected at $n = 30$) to roughly 50% by $p = 8$–12%, where the code offers essentially no protection. The mean wall-clock time per syndrome round rises mildly with p (354 ms at $p = 0.01$ to 389 ms at $p = 0.12$), consistent with the growing number of injected error gates and ancilla-reset gates dispatched per round as the syndrome becomes non-trivial more often; total sweep time was 221.7 s (excluding the pre-flight self-test) for all 600 rounds.

9 Discussion

Threshold-like behaviour. The qualitative shape of Table 2 – a sharp jump in logical error rate between $p = 1\%$ and $p = 3\%$ – is consistent, in order of magnitude, with the $\sim$1.14% accuracy threshold reported by [3]. This should not, however, be over-read as a threshold *measurement*: a genuine threshold determination requires sweeping multiple code distances and locating the p at which the logical-error-rate curves for different d cross [4, 7], whereas this study reports a single distance ($d = 3$)

at a single round count ($R = 4$) with a modest sample size (30 trials/point, giving wide confidence intervals particularly at the zero-failure point). The result presented here should be read as a qualitative, hardware-validated consistency check against the literature, not a threshold estimate.

Why the logical rate exceeds p so quickly. With $q_{\text{meas}} = p$ applied to eight ancilla readouts across three (of four) rounds, plus independent X/Z channels on nine data qubits across all four rounds, the effective per-trial error budget is substantially larger than the bare per-qubit p – by design, since this mirrors [3]'s model rather than a single-shot error rate. This explains why the logical error rate climbs above p so much faster than a naive single-round picture would suggest.

Scope limitations. (i) The CNOT scheduling within each stabilizer (Section 5) is a simple deterministic ordering, not the hook-error-optimal schedule of [5]; since this study's noise model does not inject errors during the entangling gates themselves, this does not affect the results reported here but would need addressing before extending the noise model to circuit-level (gate-time) errors. (ii) The brute-force matching decoder (Section 6) does not scale past the small event counts a distance-3 code produces; adopting PyMatching [10] or a Union-Find decoder [11] is necessary for larger distances. (iii) Only a single logical qubit and a single code distance were studied. (iv) The error model uses independent X/Z channels rather than a full depolarizing or leakage model. (v) Most importantly for an embedded-systems readership: the ∼350–390 ms measured wall time per syndrome round is dominated by host↔FPGA HBM transfer of the 17-qubit (2^{17}-amplitude) statevector and kernel-launch overhead for what is a numerically tiny circuit – several orders of magnitude slower than the sub-microsecond syndrome-to-correction latency targeted by dedicated real-time decoder pipelines such as our companion work [20]. This is expected and, for the purposes of this paper, acceptable: the goal here is exact-state *algorithmic correctness validation* of the full surface-code protocol, not real-time decoding throughput. The two goals are complementary: an exact-statevector validation harness of the kind reported here gives a verifiable ground truth that a fast, approximate, hardware-streamed decoder pipeline can be benchmarked against.

Value of the host/FPGA co-execution pattern. Beyond the specific surface-code result, Section 5's protocol – dispatch one purely unitary circuit per round, perform only the unavoidable measurement collapse and classical decoding on the host, and feed the collapsed state forward via explicit state reload – is a general pattern for validating any repeated-measurement fault-tolerance protocol on accelerator hardware that lacks native mid-circuit classical feedback, which describes a meaningful fraction of currently deployed FPGA/GPU statevector kernels, including prior systems in this same hardware family [17, 18].

10 Conclusion and future work

This paper implemented a distance-3 rotated surface code, derived and independently verified from first principles, on a 28-qubit HBM-accelerated FPGA statevector simulator, and demonstrated a host/FPGA co-execution protocol for running a genuine multi-round fault-tolerant quantum-memory experiment on hardware that offers no

native mid-circuit measurement-and-branch instruction – every entangling operation still executes on the FPGA kernel, and only the physically unavoidable measurement collapse and classical MWPM decoding run on the host. A synthetic-syndrome-data generator reproduced the Dennis–Kitaev–Landahl–Preskill error model, and a from-scratch space-time matching decoder – whose construction is reported together with a real connectivity bug we found and fixed via exhaustive self-testing – decoded 150 hardware trials (600 FPGA-executed syndrome rounds) with zero discrepancies against an independent classical cross-check, producing a logical-error-rate curve qualitatively consistent with the literature's reported accuracy threshold.

Future work includes: (i) scaling within the 28-qubit budget to larger code patches or a second logical qubit; (ii) adding a native mid-circuit ancilla measurement-and-reset primitive to the kernel itself, removing the host round-trip that currently dominates wall time; (iii) replacing the brute-force matching solver with PyMatching or a Union-Find decoder to support larger code distances; (iv) extending the noise model to full depolarizing and leakage channels; (v) cross-validating against Stim at the stabilizer-circuit level; (vi) extending to logical two-qubit operations via lattice surgery; and (vii) coupling this exact-state validation harness to the real-time FPGA decoder pipeline of [20] for an end-to-end demonstration that combines exact-state correctness validation with real-time decoding latency on the same hardware family.

Declarations

Funding. This work was carried out at the Centre for Development of Advanced Computing (C-DAC), Noida, India. *[Author: insert specific grant/project number, if applicable, before submission.]*

Conflict of interest. The author declares no conflict of interest.

Data and code availability. The FPGA host driver, surface-code circuit generator, decoder implementation, and the result data (CSV) and figure reported in Section 8 are available from the corresponding author upon reasonable request, and may be published in a public repository (e.g. https://github.com/nasir26) prior to final submission. *[Author: confirm final hosting location/DOI before submission.]*

Author contribution. N.A. conceived the study, derived and verified the surface-code lattice and decoder, implemented the host/FPGA co-execution protocol, ran the hardware experiments, and wrote the manuscript.

References

[1] Kitaev, A.Y.: Fault-tolerant quantum computation by anyons. Ann. Phys. **303**(1), 2–30 (2003)

[2] Bravyi, S.B., Kitaev, A.Y.: Quantum codes on a lattice with boundary. arXiv:quant-ph/9811052 (1998)

[3] Dennis, E., Kitaev, A., Landahl, A., Preskill, J.: Topological quantum memory. J. Math. Phys. **43**(9), 4452–4505 (2002)

[4] Fowler, A.G., Mariantoni, M., Martinis, J.M., Cleland, A.N.: Surface codes: Towards practical large-scale quantum computation. Phys. Rev. A **86**, 032324 (2012)

[5] Tomita, Y., Svore, K.M.: Low-distance surface codes under realistic quantum noise. Phys. Rev. A **90**, 062320 (2014)

[6] Versluis, R., Poletto, S., Khammassi, N., Tarasinski, B., Haider, N., Michalak, D.J., Bruno, A., Bertels, K., DiCarlo, L.: Scalable quantum circuit and control for a superconducting surface code. Phys. Rev. Appl. **8**, 034021 (2017)

[7] Google Quantum AI (Acharya, R., et al.): Suppressing quantum errors by scaling a surface code logical qubit. Nature **614**, 676–681 (2023)

[8] Chamberland, C., Zhu, G., Yoder, T.J., Hertzberg, J.B., Cross, A.W.: Topological and subsystem codes on low-degree graphs with flag qubits. Phys. Rev. X **10**, 011022 (2020)

[9] Edmonds, J.: Paths, trees, and flowers. Canad. J. Math. **17**, 449–467 (1965)

[10] Higgott, O.: PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. ACM Trans. Quantum Comput. **3**(3), 1–16 (2022) % VERIFY exact page range before submission.

[11] Delfosse, N., Nickerson, N.H.: Almost-linear time decoding algorithm for topological codes. Quantum **5**, 595 (2021)

[12] Gidney, C.: Stim: a fast stabilizer circuit simulator. Quantum **5**, 497 (2021)

[13] Javadi-Abhari, A., Treinish, M., Krsulich, K., et al.: Quantum computing with Qiskit. arXiv:2405.08810 (2024)

[14] AMD/Xilinx: Alveo U55C High-Performance Compute Card Data Sheet, DS978. AMD/Xilinx, Inc. (2023) % VERIFY current document number before submission.

[15] AMD/Xilinx: Vitis Unified Software Platform Documentation: Application Acceleration Development, UG1393. AMD/Xilinx, Inc. % VERIFY current document number before submission.

[16] AMD/Xilinx: Xilinx Runtime (XRT) Documentation. https://github.com/Xilinx/XRT

[17] Ali, N.: A Native RTL Quantum Statevector Simulator in FP64 on an HBM-Equipped FPGA. Submitted to J. Supercomput. (2026)

[18] Ali, N.: FPGA Based Quantum Circuit Simulator: Processing 30 Qubit Systems. Submitted to J. Supercomput. (2026)

[19] Ali, N.: Matrix Product State Engine for FPGA Quantum Circuit Simulation Beyond Five Hundred Qubits. Submitted to J. Supercomput. (2026)

[20] Ali, N.: FPGA-Based Real-Time Quantum Error Correction for Shor and Steane Codes. Submitted to J. Supercomput. (2026)